

Yacc

yacc规则文件

见[syntax_rule.txt](#)文件

其中主要分为两部分：

- 基本符号部分
- 生成式部分

基本符号部分

包含C语言必要终结符号，以%token开头

```
%token INT FLOAT CHAR VOID
%token IF ELSE WHILE FOR DO RETURN BREAK CONTINUE
%token ADD SUB MUL DIV MOD          /* +, -, *, /, % */
%token INC DEC                      /* ++, -- */
%token ASSIGN                       /* = */
%token EQ NE LT LE GT GE           /* ==, !=, <, <=, >, >= */
%token AND OR NOT                   /* &&, ||, ! */
%token AMPER MUL_TOKEN              /* &, * */
%token LPAREN RPAREN LBRACE RBRACE LBRACKET RBRACKET /* ( ) { } [ ] */
%token COMMA SEMICOLON              /* , ; */
%token IDENTIFIER NUMBER STRING
```

对于运算符，根据优先级从低到高按行排列，%left、%right说明运算符结合性

```
%right ASSIGN
%left OR
%left AND
%left EQ NE
%left LT LE GT GE
%left ADD SUB
%left MUL DIV MOD
%right NOT UMINUS
%right INC DEC
%left INC DEC LBRACKET LPAREN
```

%start标明文法开始符号

```
%start program
```

生成式部分

按照程序->函数->语句->表达式依次注明，详情请见文件

数据结构

Symbol

符号类型，用于记录终结符、非终结符信息

其中主要数据成员如下

- type: 终结符或非终结符
- name: 初始token字符串
- is_operator、precedence、associativity: 运算符相关，记录优先级、结合性

```
enum class Associativity
{
    Left, Right, None
};

class Symbol
{
public:
    enum class Type
    {
        Terminal, NonTerminal
    };
    ...
private:
    Type type_;
    std::string name_;
    bool is_operator_;
    unsigned int precedence_;
    Associativity associativity_;
};
```

Production

产生式类型，用于记录产生式两边符号

其中主要数据成员如下

- id: 产生式id，方便标识
- left、right: 产生式左右两边符号(集)

```
class Production
{
    ...
private:
    unsigned int id_;
```

```
Symbol left_;  
std::vector<Symbol> right_;  
};
```

yacc规则文件解析

SyntaxRuleParser类，负责读取yacc规则文件，记录到符号表、产生式表中

其中，主要数据成员如下

- inited：记录文件路径的初始化状态
- file_path：记录文件路径
- start_symbol：开始符号
- symbols：符号集
- productions：产生式集

提供以下主要函数

- init：初始化文件路径
- parse：解析文件
- show：暂时用于测试解析结果
- get_symbol：根据符号原始token字符串获取对象

```
class SyntaxRuleParser  
{  
public:  
    SyntaxRuleParser();  
  
    bool init(const std::filesystem::path& file_path);  
    bool parse();  
    void show() const;  
  
    std::optional<Symbol> get_symbol(const std::string_view name) const;  
private:  
    bool inited_;  
    std::filesystem::path file_path_;  
    Symbol start_symbol;  
    std::unordered_set<Symbol> symbols_;  
    std::unordered_set<Production> productions_;  
};
```

测试程序

设置yacc规则文件路径，输出解析结果，见syntax_generator.cpp

```
#include "syntax_rule_parser.h"  
#include <filesystem>
```

```
int main()
{
    std::filesystem::path file{"./resource/rule/syntax_rule.txt"};

    SyntaxRuleParser parser;
    parser.init(file);

    parser.parse();

    parser.show();

    return 0;
}
```

测试结果如下

```
Start:
program

=====
Symbol:
postfix_expression
cast_expression
multiplicative_expression
additive_expression
relational_expression
equality_expression
conditional_expression
assignment_expression
NOT
OR
GE
LT
function_definition
LE
selection_statement
NE
declarator
DEC
GT
MUL
statement_list
SUB
AND
external_declaration
WHILE
logical_or_expression
CONTINUE
IF
BREAK
unary_expression
```

```

INT
RPAREN
RBRACE
parameter_list
MOD
VOID
program
DIV
LBRACE
INC
expression
EQ
DO
RETURN
FLOAT
ADD
parameter_declaration
NUMBER
AMPER
initializer
iteration_statement
MUL_TOKEN
FOR
LPAREN
ELSE
LBRACKET
STRING
RBRACKET
IDENTIFIER
declaration
UMINUS
COMMA
external_declaration_list
ASSIGN
type_specifier
logical_and_expression
':'
init_declarator_list
jump_statement
init_declarator
compound_statement
statement
'?'
CHAR
SEMICOLON
expression_statement

=====
Production:
76 unary_expression -> DEC unary_expression
75 unary_expression -> INC unary_expression
74 unary_expression -> postfix_expression
73 cast_expression -> LPAREN type_specifier RPAREN cast_expression
72 cast_expression -> unary_expression

```

```
71 multiplicative_expression -> multiplicative_expression MOD cast_expression
70 multiplicative_expression -> multiplicative_expression DIV cast_expression
69 multiplicative_expression -> multiplicative_expression MUL cast_expression
68 multiplicative_expression -> cast_expression
67 additive_expression -> additive_expression SUB multiplicative_expression
66 additive_expression -> additive_expression ADD multiplicative_expression
65 additive_expression -> multiplicative_expression
64 relational_expression -> relational_expression GE additive_expression
63 relational_expression -> relational_expression GT additive_expression
62 relational_expression -> relational_expression LE additive_expression
61 relational_expression -> relational_expression LT additive_expression
60 relational_expression -> additive_expression
59 equality_expression -> equality_expression NE relational_expression
28 statement -> iteration_statement
27 statement -> selection_statement
26 statement -> compound_statement
25 statement -> expression_statement
24 parameter_declaration -> type_specifier
23 parameter_declaration -> type_specifier declarator
22 parameter_list -> parameter_list COMMA parameter_declaration
21 parameter_list -> parameter_declaration
20 function_definition -> type_specifier IDENTIFIER LPAREN RPAREN
compound_statement
19 function_definition -> type_specifier IDENTIFIER LPAREN parameter_list RPAREN
compound_statement
18 initializer -> expression
17 declarator -> declarator LBRACKET RBRACKET
16 declarator -> declarator LBRACKET NUMBER RBRACKET
15 declarator -> MUL_TOKEN declarator
14 declarator -> IDENTIFIER
13 init_declarator -> declarator ASSIGN initializer
0 program -> external_declaration_list
1 external_declaration_list -> external_declaration
2 external_declaration_list -> external_declaration_list external_declaration
3 external_declaration -> function_definition
4 external_declaration -> declaration
5 declaration -> type_specifier init_declarator_list SEMICOLON
6 type_specifier -> INT
7 type_specifier -> FLOAT
8 type_specifier -> CHAR
9 type_specifier -> VOID
10 init_declarator_list -> init_declarator
11 init_declarator_list -> init_declarator_list COMMA init_declarator
12 init_declarator -> declarator
29 statement -> jump_statement
30 statement -> declaration
31 expression_statement -> expression SEMICOLON
32 expression_statement -> SEMICOLON
33 compound_statement -> LBRACE statement_list RBRACE
34 compound_statement -> LBRACE RBRACE
35 statement_list -> statement
36 statement_list -> statement_list statement
37 selection_statement -> IF LPAREN expression RPAREN statement
38 selection_statement -> IF LPAREN expression RPAREN statement ELSE statement
```

```
39 iteration_statement -> WHILE LPAREN expression RPAREN statement
40 iteration_statement -> DO statement WHILE LPAREN expression RPAREN SEMICOLON
41 iteration_statement -> FOR LPAREN expression_statement expression_statement
expression RPAREN statement
42 iteration_statement -> FOR LPAREN expression_statement expression_statement
RPAREN statement
43 jump_statement -> RETURN expression SEMICOLON
44 jump_statement -> RETURN SEMICOLON
45 jump_statement -> BREAK SEMICOLON
46 jump_statement -> CONTINUE SEMICOLON
47 expression -> assignment_expression
48 expression -> expression COMMA assignment_expression
49 assignment_expression -> conditional_expression
50 assignment_expression -> unary_expression ASSIGN assignment_expression
51 conditional_expression -> logical_or_expression
52 conditional_expression -> logical_or_expression '?' expression ':'
conditional_expression
53 logical_or_expression -> logical_and_expression
54 logical_or_expression -> logical_or_expression OR logical_and_expression
55 logical_and_expression -> equality_expression
56 logical_and_expression -> logical_and_expression AND equality_expression
57 equality_expression -> relational_expression
58 equality_expression -> equality_expression EQ relational_expression
```